



Java Singleton

The purpose of Singleton is to enforce at most one instance of some class and to control access to it.

The "classic" example...

```
public class LogManager
{
    private java.io.PrintStream m_out;

    private LogManager( PrintStream out )
    {
        m_out = out;
    }

    public void log( String msg )
    {
        m_out.println( msg );
    }
    static private LogManager sm_instance;
    static public LogManager getInstance()
    {
        if ( sm_instance == null )
            sm_instance = new LogManager( System.out );
        return sm_instance;
    }
}
```

Usage:

```
LogManager.getInstance().log( "some message" );
```

It should be noted that this is a good naive solution. In the general case, however, it is not thread-safe.

It's possible in a naive implementation for one thread to preempt after the test for null but before the instance creation, so that the first thread (which has already tested for null) calls the constructor again and creates another instance. You could just synchronize the `getInstance()` method but it's not efficient, what you really want to do is just synchronize the instance creation, like so:

```
public class Singleton {

    private Singleton() {
    }

    public static Singleton getInstance() {
        if (_theInstance == null) {
            synchronized (Singleton.class) {
                if (_theInstance == null) {
                    _theInstance = new Singleton();
                }
            }
        }
    }
}
```

```

        }
        }
        return _theInstance;
    }
}

```

Following up myself: It turns out that this will not work reliably. This issue turns out to have become a major point of discussion and resulted in The "Double-Checked Locking is Broken" Declaration:

<http://www.cs.umd.edu/~pugh/java/memoryModel/DoubleCheckedLocking.html>

[DoubleCheckedLockingIsBroken](#)

-- [StevenNewton](#)

You are correct. The simplest approach is to do the following:

```

public class Singleton {
    private static final Singleton _theInstance = new Singleton();

    private Singleton() {
    }

    public static Singleton getInstance() {
        return _theInstance;
    }
}

```

The Java spec guarantees that the static initialiser will be executed only once, at class load time.

--[DarrenHobbs](#)

This last approach seems to work well in most cases -- and perhaps it is supposed to work per the Java spec. But -- as I've recently learned the hard way -- both Tomcat and JBoss do not always execute this code correctly. In particular, if some code is placed in the body of the constructor, even something trivial like logging to System.out, it is not always executed. I've also had initialization of member variables silently fail in the constructor. Unfortunately, deadlines have driven me to seek workarounds rather than research this situation. I'd love to learn more about it, even if that means learning I've made some big dumb mistake in what I was trying!

- Adam

Could you provide some example code for which Tomcat or JBoss fails to execute code placed in the body of the constructor, possibly under a new topic?

Also, the above approach doesn't work with constructors that throw exceptions.

In that case you could do the initialization inside a try block inside a static block. If the singleton is going to throw an exception during initialization, then it's better to do it as soon as possible one would suppose. Of course, classloading time for a singleton is normally the same time as the first call to getInstance anyway, so you'll get your exception...

This approach works fine. One thing that needs to be added in this code is to check if there was a problem in creation of the singleton. If that is not handled you'll encounter unexpected `NullPointerException`?

```
public class Singleton {  
    private static final Singleton _theInstance = new Singleton();  
    private Singleton()  
    {  
        try {}  
        catch (Exception e) {}  
    }  
    public static Singleton getInstance()  
    {  
        if (_theInstance == null) {  
            throw new RuntimeException("No singleton instance available");  
        } else {  
            return _theInstance;  
        }  
    }  
}  
  
--AJ
```

Could this be to do with your Tomcat configuration? I think that you have to specifically configure Tomcat to only create a single instance of a Servlet object. I've been trying to do this myself, this webpage explains Tomcat configuration options:

<http://www.onjava.com/pub/a/onjava/2002/07/31/tomcat.html?page=2>

Extract: *crossContext*: When set to true, allows the `ServletContext`?`.getContext()` method to successfully return the `ServletContext`? for other Web applications running in the same host. The default value is false, which will prevent the access of cross-context access.

I haven't tested this yet, but somebody much cleverer than me said that it should work...

- Dan J

Here's an elegant variation on Darren Hobbs' code above. It's quoted by kind permission of Scot Floess, who in turn credits Josh Bloch's "Effective Java" for some form of it. Scot outlines it in a letter to JavaWorld here: <http://www.javaworld.com/javaworld/jw-05-2003/jw-0530-letters.html>

```
public final class Singleton {  
    private static final class SingletonHolder {  
        static final Singleton singleton =  
            new Singleton();  
    }  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        return SingletonHolder.singleton;  
    }  
}
```

}

(Blame me for messing slightly with Scot's code ;-) Paraphrasing Scot, the point is that merely using a static to hold a singleton will mean the singleton gets constructed if someone refers to the containing class. This is especially significant if the singleton is very heavyweight to construct. An embedded static final class can defer this, as shown. If someone happens to refer to Singleton (or even Singleton.class), the singleton will not be created unless there is a call to getInstance(). At that time, SingletonHolder is referred to; its class loads; and its static member (namely singleton) is instantiated. Arguably, it is unlikely that code refers to Singleton without calling getInstance(). But it will defer singleton construction until it is absolutely required.

I asked Scot about thread-safety - note that the code doesn't use synchronized anywhere - and he replied, "... this is all very thread safe. Specifically, when the VM attempts to load a class, you are guaranteed that while loading the class, no other thread will be attempting to 'muck' with the class being loaded (nor will the same class loader attempt to load the same class). This makes perfect sense if you consider class loading to be of the same nature as object instantiation."

If one thing's clear from the widespread discussion of singleton it's that any implementation should be matched carefully to the application at hand; the term singleton is subject to interpretation depending on context. With that caveat in mind, though, the above is a clean and elegant solution in many cases.

-- [LloydBlythen](#)

"Using the Singleton Pattern" article by Budi Kurniawan 08/27/2003 <http://www.onjava.com/pub/a/onjava/2003/08/27/singleton.html> ... Some comments at the end of the article point here to [WardsWiki](#).

Unfortunately the above article is a good example of how to implement Singleton _incorrectly_ - especially its Listing 1.

-- [LloydBlythen](#)

Actually, I wonder since Java has Class Objects, can we find ways to avoid the traditional singleton implementation? After all, a Class object is a *natural singleton*. If this is true, it should follow that in Java it is more natural to represent a singleton as a Class Object? Of course, Java's Class is an odd sort of Meta Classes, it cannot be subclassed. Still, in many cases, using the Class Object is a much better solution for Java than a GoF-style singleton. In those cases where a Class Object will not do (even with a static initializer), one can instead avoid the [SingletonPattern](#) by using a context object passed around between consumers, much like the TestContext class described in [CppUtxOverview](#). To me, Singleton has always seemed like a way to feel good about GlobalData? and I think Java provides us more opportunities to eradicate its use. --[RobertDiFalco](#)

For simple singletons, doing it this way (using statics) is definitely the right way to go. There are times when you might actually want to vary the implementation of the singleton between invocations (e.g., a Log singleton might be an instance of FileLogger in one environment, and an instance of JiniLogger in another). You can still do that by making it appear to be the class, but that seems (to my mind) to obscure what's really going on a little too much. --[GlennVanderburg](#)

In those cases you can simply wrap the class instance with static members.

```
public interface Logger
{
```

```

        void out( Object msg );
    }

---

public class SystemLogger?
{
    static private Logger sm_logger;
    static
    {
        sm_logger = getConcreteLoggerFromProperties();
    }

    public static void out( Object msg )
    {
        sm_logger.out( msg );
    }
}

```

Or something like that. Of course, I think you are right. The Class reference a static instance of itself probably isn't a problem one needs to think about too much. However, this does a good job of explicitly showing this is a System-Wide usage of the *Logger* interface. -- [RobertDiFalco](#)

[RobertDiFalco](#) wrote: *After all, a Class object is a natural singleton. If this is true, it should follow that in Java it is more natural to represent a singleton as a Class Object...*

I disagree. There are *three* reasons for Singleton, only one of which is that you only want one instance. Reason two is that you want to make that instance easy to find (which the Class approach also does well at).

Consider that reason two is not a sufficient call for Singleton without reason one. Often, merely having a global name for one instance suffices rather than creating a Singleton.

But the *third* reason is that you don't want to lock in your decision to make only one instance. A common design shift is to later want one instance per thread, or per user, or per something that there's definitely more than one of. If you've implemented everything as static methods on a class, how are you going to do that without modifying client code? (OK, for per-thread, this is possible because Java implicitly passes around the current thread; but in the general case, you'd have to change the interface and all the calls.)

Another shift is simply that you don't want lower-level objects to know about the Singleton-ness of the instance (see Andrew's comments on [SingletonsAreEvil](#)). The lack of this protective ignorance is what makes Singleton take on much of the evil of Globals (which many people have commented on). But the ignorance-enabled flexibility comes at a price: it's now a little more complicated to let the clients know where to find a/the instance they need. A [ContextObject](#) is a good, popular point along this tradeoff continuum.

--[GeorgePaci](#)

[GeorgePaci](#) wrote: *But the third reason is that you don't want to lock in your decision to make only one instance. ... If you've implemented everything as static methods on a class, how are you going to do that without modifying client code?*

I think it is possible by implicitly referring to an instance. Consider the examples below:

a)

```

public class Logger {

```

```

    public static void out( Object msg ) {
        print( msg ); // static
    }
}
Logger.out( "hiya" );

```

b)

```

public class Logger {
    private static Logger getInstance () { ... }
    public static void out( Object msg ) {
        getInstance().print( msg );
    }
}
Logger.out( "hiya" );

```

Also compare b) to the classical implementation c). I think it is pretty much similar, since `getInstance()` in the classic approach is always a class method itself.

c)

```

public class Logger {
    public static Logger getInstance () { ... }
    public void out( Object msg ) {
        print( msg );
    }
}
Logger.getInstance().out( "hiya" );

```

--[MartinSchwartz](#)

[EdShea](#) of [WorldStreet](#) discovered this addition to the [JavaPatterns...](#)

The Singleton implementation suggested by the [GangOfFour](#) has an interesting failure mode in Java 1.1 (Java 1.2 and later do not exhibit this problem).

If the sole reference to the singleton is from the class, then the java 1.1 collector will collect the instance and unload the class. The next retrieval of the singleton will reload the class and manufacture a newly initialized singleton instance.

The solution is to register singleton instances in some system wide table so that it is protected from garbage collection. Consider using latent threads:

Create a thread for each singleton at the time that the singleton instance is created. Do not schedule the thread to run, this is not the purpose of the thread. The mere existence of the thread will protect the singleton. Also consider using a registry class which maintains a collection of the singleton instances. Create one latent thread for the registry instance. Both the registry and any singleton instances it has references to will be protected from garbage collection and it only uses one thread.

See [AnotherJavaSingletonProblem](#) and [InheritedJavaSingletonProblem](#)

I think its important to point out that this problem only exists for Java 1.1 - The [GangOfFour](#) implementation of a singleton is perfectly correct in the later implementations of gc. In later implementations of Java the problem was resolved by only gc'ing if there were zero references to the static class including itself. To

implement this correctly if you were expecting to run your java code on 1.1 you could just check the version of java and implement the threading approach only if necessary.

just 2 pence!

-- Lee Harris

Alternatively, store the instance in a system property when it's first instantiated, as a pre-built registry.

-- [MartinPool](#)

Is this an actual problem? After all, if only the class holds a reference to the singleton (for *technical* reasons), who would notice if it was garbage collected and replaced by a different singleton? How? --[HaskoHeinecke](#)

How about if the singleton implemented a counter? If the singleton was replaced by a new instance, the counter would reset. --[SteveFreeman](#)

I begin to understand: You are referencing the singleton's class in your code and the class references the singleton. You would expect this indirect reference to be enough to preserve the singleton and its state. (And you would probably be right if classes were first class objects.) BUT your reference to the *class* is weak with respect to the GC, so the singleton's class is thrown away, and consequently the singleton. Interesting.

--[HaskoHeinecke](#)

You could conceivably use the finalize method to serialize your singleton into a file or database (a la bean managed persistence), and then reload the object in the getInstance method.

This way you get both long term persistence and more efficient memory management. Does this make sense?

--GadiGuy? (gadi-g@actcom.co.il)

I wish the JavaDeities? would just add a *single* keyword so every instance created (with *new*) by the JVM refers to to same object. This would make it trivial to code:

```
public single class OneClass {
    public static OneClass getInstance() { return new OneClass(); }
}
```

Not that the [JavaSingleton](#) is hard to code... the standard implementation is just an annoying idiom (to me). --[JimmyCerra](#)

As of release 1.5, there is an excellent approach to implementing singletons. Simply make an enum type with one element:

```
// Enum singleton - the preferred approach
public enum Elvis {

    INSTANCE;

    public void leaveTheBuilding() { ... }
}
```

}

This is by far the best and reliable way to implement singletons.

Refer to 'Effective Java' book

See also: [SingletonInUmlForJava](#), [InheritedJavaSingletonProblem](#), [AnotherJavaSingletonProblem](#), [JavaStaticProblem](#), [JavaSingletonProblems](#)

[CategoryJava](#)



[EditText](#) of this page (last edited [April 30, 2012](#)) or [FindPage](#) with title or text search